FileNest: Distributed Semantic Retrieval over Decentralized Data

Aradhya Mahajan
IIT Jodhpur

Lakshya Jain
IIT Jodhpur

Susil Kumar Mohanty
IIT Jodhpur

Abstract

Decentralized, content-addressed storage systems such as IPFS let any peer that holds a copy of an object serve it, but finding content still depends on search services run by a single operator, which can filter, delist, or reorder what users are able to find. FileNest is a peer-to-peer system that provides semantic search over such storage without a central index. Each content object is mapped to an embedding, and the embeddings are organized in a hierarchy of tagging vectors: routing points in the embedding space that are created as content arrives, under similarity thresholds that tighten with depth. A distributed hash table (DHT) assigns each tagging vector to a responsible peer by a hash of its value at creation, which separates semantic routing, determined by vector similarity, from physical placement, determined by the DHT. A query descends the same hierarchy, and a covering radius maintained for each tagging vector lets peers skip subtrees that provably contain no match, so pruning never discards a result. We define the system model, the hierarchy and its threshold schedule, responsibility assignment, and the indexing and query procedures, and we extend the design with replicated responsibility, pre-indexing deduplication, and recall-calibrated pruning, which trades a calibrated recall target for fewer query messages.

1 Introduction

Semantic retrieval represents content using properties of its underlying information rather than relying solely on lexical identifiers such as filenames or manually assigned labels. This representation is useful for collections containing heterogeneous objects, including text documents, images, videos, and other data types. A semantic retrieval system can represent such objects as vectors produced by a learned encoder [32, 45, 49] and perform retrieval according to a similarity function defined over the resulting vector space.

A conventional semantic search architecture maintains a centralized index containing representations of the searchable collection [30, 31, 37]. This simplifies index construction and query processing because a single service maintains the searchable state. In a distributed setting, however, content may be stored by different devices or organizations. A centralized index must then maintain information about content residing outside the indexing service.

Centralizing the index also centralizes control over what can be found. Search engines determine which content is reachable in practice, giving prominence to some sources at the expense of others [28], and the order of their results influences what users believe: in controlled experiments, biased rankings shifted the voting preferences of undecided voters by 20% or more [20]. Web search is also highly concentrated. Google handles roughly 90% of search queries worldwide [51]; in 2024 a U.S. federal court held that it is a monopolist in general search services, with 89.2% of U.S. general search queries and 94.9% of those on mobile devices [57], and the European Commission fined it €2.42 billion for abusing its dominance as a search engine by favoring its own comparison-shopping service in its results, a decision upheld by the Court of Justice in 2024 [11, 21].

A centralized index is also the point at which results are removed. Search operators can be required by law to delist results for searches on a person's name, even when the underlying page remains lawfully published [10]. In 2002, Google excluded more than a hundred sites, in whole or in part, from its French and German services [62], and censored results on google.cn from its launch in 2006 until 2010, when it stopped and redirected users to Hong Kong [18, 19]. Microsoft's Bing has suppressed the names of Chinese political dissidents from its autosuggestions for users in the United States and Canada [33], and on the anniversary of the 1989 Tiananmen Square protests returned no image results for "tank man" in the United States and elsewhere, which Microsoft attributed to human error [12]. Whatever the merits of an individual removal, it is applied at one point and takes effect for every user of that index. Censorship of web content is also widespread at the network level. Measurement platforms have observed the blocking of thousands of URLs in 60 countries and growing censorship activity in more than 100 [22, 41, 46],

and Freedom House estimates that 69% of the world's internet users live in countries where political, social, or religious content was blocked online [23].

Decentralized, content-addressed storage removes the single point of control for storage. In IPFS, any peer that holds a copy of an object can serve it, and the object is named by a hash of its content, so a copy can be verified regardless of where it was obtained [3]; the public network spans more than 2,700 autonomous systems in 152 countries [56]. When Turkey blocked all language editions of Wikipedia in April 2017, a block lifted only in January 2020 after the Constitutional Court found that it violated freedom of expression [60], the IPFS project published a snapshot of Turkish Wikipedia on IPFS that any peer holding a copy could serve [55]. The mirror also shows the limit of storage alone. It offered no full-text search, only a client-side lookup of article titles, and later versions of the mirror had no search function at all [47]. Retrieval in IPFS begins from a content identifier [56], and such identifiers are not human-readable, so naming remains an open problem in these networks [14]. The designers of Freenet, an earlier censorship-resistant store, identified the same gap: retrieval by key "still leaves the problem of finding keys in the first place" [9]. Search services built for IPFS have filled it centrally. The ipfs-search service, for example, discovered content by monitoring DHT traffic and indexed it in a single Elasticsearch, later OpenSearch, backend, and its public service has been offline since 2023 [29]. Such services reintroduce at the search layer the single point of control that decentralized storage removes at the storage layer.

FileNest is motivated by the goal of an open web in which the ability to find content, and not only to store and serve it, does not depend on a small number of companies or on any single operator. It provides search over decentralized storage without a central index: the index is partitioned among the participating peers, no single peer holds or controls it, and content is found by its meaning rather than by an identifier or keyword that the user must already know. Semantic retrieval suits this setting, because content published independently by many parties shares no common vocabulary or metadata conventions. FileNest does not by itself defeat network-level blocking or shutdowns, of which Access Now recorded 313 in 52 countries in 2025 [1]. It removes the index as a point of control, so that content that remains reachable at the storage layer also remains findable.

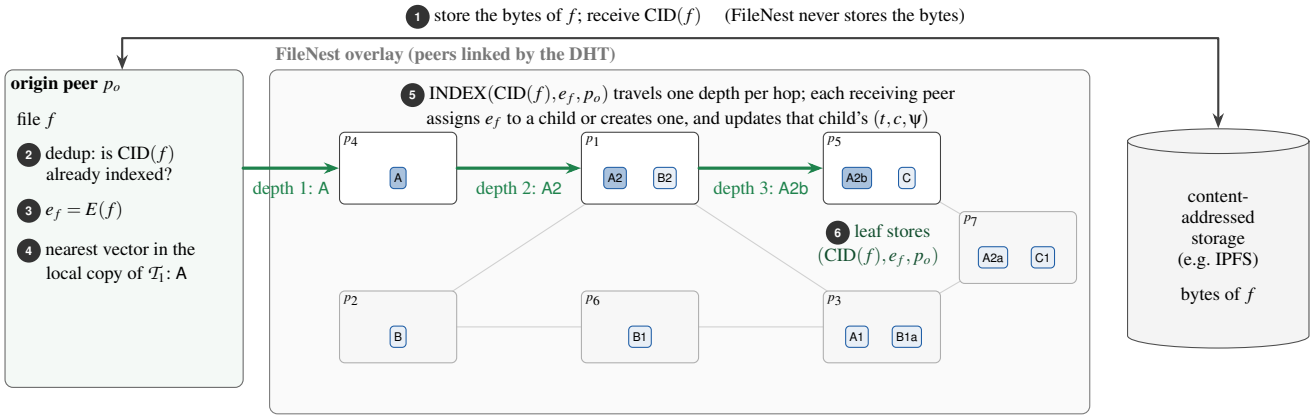
FileNest indexes content held in an external content-addressed storage layer such as IPFS [3]; the index itself holds only content identifiers and embeddings. A shared encoder maps every content object and every query to an embedding, and FileNest organizes the embeddings in a hierarchy of *tagging vectors*: routing points in the embedding space, each the running mean of the embeddings assigned to it. At each depth, an embedding joins its nearest tagging vector if that vector lies within a threshold that tightens with depth, and otherwise creates a new one. A distributed hash table (DHT) [38, 52] assigns each tagging vector to a responsible peer, keyed by a hash of the vector's value at creation. Semantic similarity thus decides *which* region an embedding belongs to, and the DHT decides *where* that region's state is kept. A query descends the same hierarchy. Each tagging vector maintains a covering radius that bounds the angle to every embedding below it, and a subtree is explored only if its covering region can intersect the query's match radius, so by the triangle inequality pruning never discards a match. Figure 1 shows both operations end to end.

The remainder of this paper is organized as follows. Section 2 surveys related work. Section 3 introduces angular distance and the DHT operations used throughout. Section 4 defines the system model, the tagging-vector hierarchy, responsibility assignment, and the indexing and query procedures. Section 5 extends this design with replicated responsibility, pre-indexing deduplication, and recall-calibrated pruning. Section 6 reports the experimental evaluation, and Section 7 discusses security and privacy considerations. Section 8 concludes the paper and outlines future work.

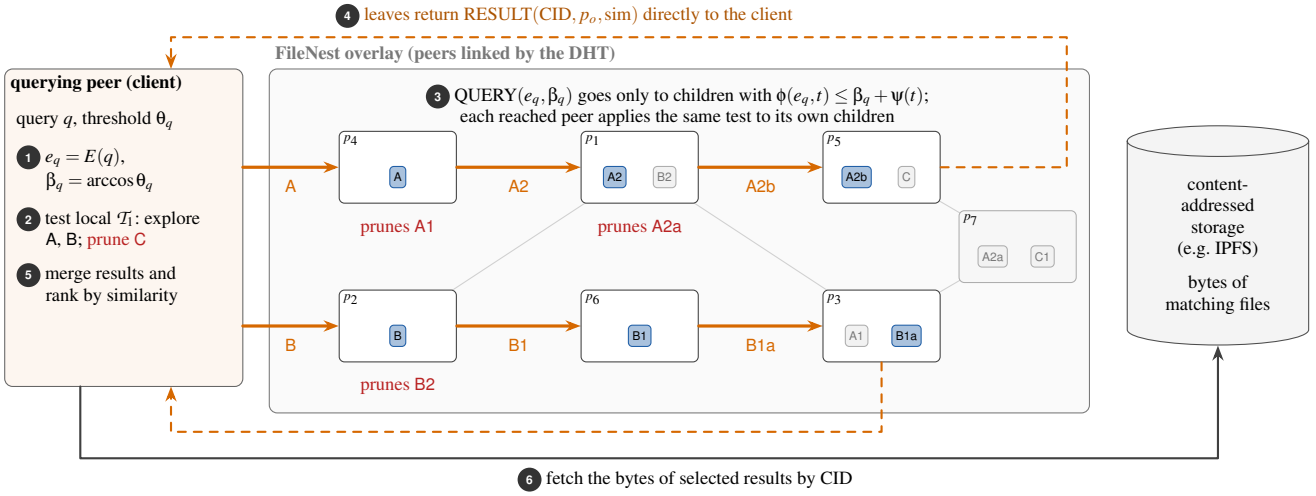
2 Related Work

FileNest builds on structured peer-to-peer overlays and content-addressed storage, which provide its placement and storage layers. It also builds on similarity search in metric spaces and incremental clustering, from which its routing hierarchy is derived. We review these areas below, together with censorship-resistant publishing systems, prior search systems built over peer-to-peer networks, and recent decentralized search engines that address result integrity and incentives.

Structured overlays and replication. A distributed hash table (DHT) assigns every key to a responsible peer and locates that peer in a bounded number of overlay hops, without requiring any peer to know the full membership. Chord places peers and keys on an identifier ring and resolves a lookup in $O(\log N)$ messages using finger tables [52]. CAN divides a d-dimensional Cartesian coordinate space into zones, one per peer, and routes in $O(dN^{1/d})$ hops [48]. Kademlia measures distance between identifiers with the XOR metric and issues lookups to several peers in parallel, so that unresponsive peers do not stall a lookup [38]. Storage systems built on these overlays survive peer departures by replicating each item on a small set of peers determined by its key: the k successors of the key on the Chord ring in CFS [13], the k peers with numerically closest identifiers in PAST [50], and the k peers closest under the XOR metric in Kademlia [38]. Cluster file systems such as GFS reach the same goal differently: a central master places each chunk on several chunkservers [24]. FileNest uses a DHT only to assign responsibility for routing state. Its replication mechanism (Section **??**) reuses the key-derived replica sets of the DHT-based systems rather than a coordinator, and



(a) Indexing a file f



(b) Resolving a query q

Figure 1: FileNest in operation, on a three-depth hierarchy whose tagging vectors (labeled chips) are spread over peers $p_1, \ldots, p_7$. (a) Indexing: the origin peer stores the bytes of f in the external content-addressed storage layer and keeps only CID(f) (1), skips content that is already indexed (2), computes the embedding (3), and selects a depth-1 tagging vector from its local copy of $\mathcal{T}_1$ (4). The INDEX message then makes one hop per depth, each to the peer responsible for the tagging vector chosen at that depth (5), and the leaf stores the entry (6). (b) Querying: the client embeds the query (1) and tests $\mathcal{T}_1$ locally (2); QUERY messages reach only subtrees that pass the pruning test (3); leaves return their matches directly to the client (4), which ranks them (5) and fetches the bytes of selected results from the storage layer (6). Gray chips are pruned, and p_7 is never contacted. Panel (b) shows the query of Figure 7.

extends them from stored items to tagging-vector state.

Content-addressed storage and deduplication. IPFS combines a Kademlia-derived DHT with content addressing: objects form a Merkle DAG whose nodes are named by cryptographic hashes, and the DHT records which peers provide each block [3]. Naming data by its hash makes duplicates detectable by equality of names. LBFS chooses chunk boundaries with Rabin fingerprints, so that boundaries survive local edits, and transmits only chunks whose hashes the receiver does not already hold [39]. Farsite applies the same principle to whole files in a serverless distributed file system, coalescing files that have identical content hashes [16]. These systems retrieve an object only when its identifier is already known. They also remove redundant storage and transfer, but not redundant indexing: two peers that submit the same file to IPFS obtain the same identifier, yet an indexing layer above it would still embed and insert the file twice. FileNest relies on a content-addressed layer for storage (Section 4.1), and its pre-indexing deduplication step (Section **??**) applies whole-file deduplication, keyed on the storage layer's identifiers, to the index itself.

Censorship-resistant publishing. Systems predating IPFS aimed to keep published content available against attempts to remove it. Freenet stores files across peers without a centralized location index, so that an adversary cannot remove a file by attacking the nodes that hold it, and protects the anonymity of both publishers and readers [9]. Free Haven provides a distributed anonymous storage service [15]. Publius encrypts each document, splits the key with Shamir secret sharing, and stores the ciphertext and one share on each of several servers, so that the document can be recovered through a Publius URL even if some servers are unavailable [59]. Tangler makes newly published documents depend on the stored blocks of previously published ones, so that a block serves several documents and publishing a document replicates the content of others [58]. These systems protect stored content but retrieve it only by a key or address that the reader already holds. Freenet's designers defer search to future work because a web-style crawler would conflict with the goal of avoiding centralization [9]. FileNest addresses this problem for content-addressed storage: it assumes such a storage layer and provides search over it without a central index.

Search over peer-to-peer networks. Several deployed systems provide search over peer-to-peer networks without semantic representations. YaCy is a peer-to-peer web search engine in which peers crawl pages into a local reverse word index and store each entry at the peers whose identifiers are closest to the hash of its word, so that a query is resolved by contacting the peers closest to the hashes of its terms [27]. Tribler extends BitTorrent with decentralized content discovery and recommendation driven by social relations among peers [44], and records peer contributions in TrustChain, a tamper-proof ledger from which free-riders can be identified [43]. Sonar replicates content collections between peers over the Hypercore protocol and indexes them locally with the Tantivy full-text engine [2]. These systems locate content through terms, metadata, or social signals. They cannot retrieve content that shares no terms with the query, and they index non-textual content only through its textual metadata. FileNest instead matches queries by similarity in a shared embedding space, which places paraphrases, and content of different modalities, near each other if the embedding function does.

Content-based and semantic routing. Other systems route queries by a summary of content rather than by exact keys. Koloniari and Pitoura summarize the XML documents held by each peer with multi-level Bloom filters and organize peers hierarchically by the similarity of their summaries, so that a path query is forwarded only toward peers whose summaries admit a match [34]. A Bloom filter answers membership queries on exact items, with a false-positive rate that follows in closed form from the filter's parameters [5, 6]. It does not measure how close a query is to the summarized content. pSearch is closer to FileNest in representation: it describes documents by semantic vectors computed with latent semantic indexing and uses these vectors as coordinates in a CAN overlay, so that a document's index entry is stored in the zone that contains its vector, and a query is routed to the point given by its own vector and then forwarded to nodes within a search radius around it [53]. In pSearch the semantic vector is therefore also the placement key, whereas FileNest places a region by a hash of its tagging vector at creation, not by its position (Section 4.3).

Similarity search in metric spaces. Metric trees prune a search with covering radii. In the M-tree, every routing object stores the radius of a ball that contains its subtree, and the triangle inequality excludes a subtree from a range query when that ball and the query ball are disjoint [8]. The cover tree arranges its levels so that covering radii shrink by a constant factor from each level to the next [4]. FileNest applies covering-radius pruning to the angular metric (Section 4.7) and derives its threshold schedule from the same geometric principle (Section 4.4). Unlike these trees, it places each node of the hierarchy at a different peer and maintains covering radii incrementally while the tagging vectors drift. Distributed metric indexes have adopted the same pruning idea. M-Chord maps objects to one-dimensional keys with an iDistance-style pivot mapping and distributes key ranges over Chord [42]. SIMPEER lets each peer cluster its own data and publish each cluster as a centroid and radius to a super-peer, which forwards a range query only to peers whose clusters intersect the query ball [17]. In both systems, placement follows from the content itself: from the mapped key in M-Chord, and from the peer that holds the data in SIMPEER. FileNest instead creates regions incrementally by a threshold rule and assigns each region to a peer through the DHT.

Approximate variants of these methods give up completeness to reduce cost. PAC nearest-neighbor search answers queries within a user-specified accuracy and confidence, using the distance distribution estimated from the data [7]. Inverted-file vector indexes probe only the few coarse cells nearest the query [30]. FileNest's recall-calibrated pruning (Section **??**) belongs to this family. It shrinks the covering radius used in the pruning test and calibrates the shrink factor empirically against the exact procedure, which requires no relevance labels.

Embeddings and vector search. Transformer encoders produce dense embeddings for semantic retrieval of text [32, 49], and contrastively trained encoders map images and text into a shared embedding space [45]. Vector search over such embeddings relies on quantization-based inverted files [30], proximity graphs such as HNSW [37], and GPU-accelerated libraries such as Faiss [31]. All of these assume that a single operator holds the index. FileNest treats the embedding function as

a black box and leaves the index used locally at a leaf unspecified. Its subject is the distribution of the index across autonomous peers.

Incremental clustering. FileNest's creation rule assigns an embedding to the nearest existing center within a threshold, and otherwise makes the embedding a new center. The leader algorithm assigns each point to the first existing leader within a threshold and otherwise makes the point a new leader [26]; FileNest applies its nearest-center variant at each depth, with a threshold specific to that depth. BIRCH similarly builds a tree of cluster summaries incrementally, and starts a new leaf entry when absorbing a point would exceed a threshold [61]. The tagging-vector update is the sequential mean update used in MacQueen's formulation of k-means [36]. These algorithms run on a single machine. FileNest assigns each center to its own responsible peer, and so must fix each center's DHT key at creation even though the center keeps moving (Section 4.3).

Integrity and incentives in decentralized search. Recent decentralized search engines address threats that FileNest does not. DeSearch runs the stages of a search pipeline inside Intel SGX enclaves operated by untrusted executors, and records witnesses that allow any party to verify that each stage executed correctly [35]. SwarmSearch, designed for deployment in Tribler, gives each peer a generative retrieval model that maps queries directly to document identifiers, following the differentiable search index [54] and its decentralized variant De-DSI [40]. A querying peer aggregates candidates from several peers and reranks them with a decentralized learning-to-rank model trained on click feedback, and donations are distributed to peers in proportion to their past contributions. The authors report that ranking quality is maintained with up to half of the peers adversarial [25]. FileNest provides no comparable guarantee for results reported by responsible peers, and no incentive for hosting routing state. Both concerns are orthogonal to the routing and placement questions studied in this paper.

In summary, prior peer-to-peer search systems either match exact terms or paths, or derive the placement of index state from the content itself: from its position in the semantic space or from the peer that holds it. Prior metric indexes provide the pruning machinery that FileNest uses, but assume a single site. FileNest combines covering-radius pruning over an incrementally built hierarchy with DHT-based assignment of each region to a peer, so that semantic routing and physical placement remain independent.

3 Background and Preliminaries

This section introduces the distance FileNest uses to compare embeddings and the two DHT operations on which its placement relies.

3.1 Angular Distance

FileNest compares embeddings and tagging vectors by cosine similarity, both during indexing and during query processing. For vectors $u, v \in \mathbb{R}^n$,

$$\mathrm{sim}(u,v) = \frac{u^T v}{\|u\|_2 \|v\|_2}.$$

We state thresholds in terms of the angular distance

$$\phi(u,v) = \arccos \mathrm{sim}(u,v), \qquad \phi(u,v) \in [0,\pi],$$

the angle between u and v. Because cos is strictly decreasing on $[0,\pi]$, a similarity threshold θ and the angular threshold $\alpha = \arccos\theta$ express the same condition,

$$\mathrm{sim}(u,v) \geq \theta \quad \Longleftrightarrow \quad \phi(u,v) \leq \alpha,$$

with a smaller angle corresponding to a larger similarity. Angular distance is the geodesic distance between $u/\|u\|_2$ and $v/\|v\|_2$ on the unit sphere and is therefore a metric on directions; in particular, for nonzero $u, v, w \in \mathbb{R}^n$ it satisfies the triangle inequality

$$\phi(u,w) \leq \phi(u,v) + \phi(v,w).$$

Cosine similarity, and the dissimilarity $1 - \mathrm{sim}$, satisfy no such inequality. We therefore state the bounds of Sections 4.6 and 4.7 in angles and convert them to similarities only at the point of comparison.

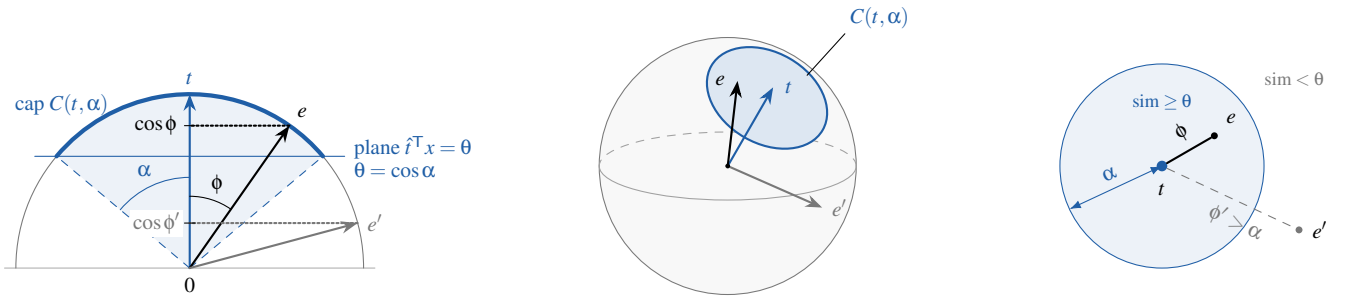
Figure 2 illustrates these quantities, and Figures 4–7 use its chart view. No flat chart of the sphere is exact, but every argument these figures depict uses only the triangle inequality, which holds on the sphere in any dimension n just as it does on the page.

3.2 Distributed Hash Tables

A distributed hash table (DHT) assigns every key in a key space to a responsible peer [38, 52]. Each peer p has an identifier $\mathrm{PID}(p)$ in the same space, and the DHT's distance metric, such as clockwise distance on the Chord ring or the XOR metric of Kademlia, determines which peers are closest to a given key. We write $\mathrm{DHTLookup}(\kappa)$ for the peer whose identifier is closest to key κ, and $\mathrm{DHTLookup}_{k_r}(\kappa)$ for the ordered list of the k_r closest peers, nearest first, so that $\mathrm{DHTLookup}_1(\kappa) = (\mathrm{DHTLookup}(\kappa))$. A lookup completes in a number of overlay hops logarithmic in the number of peers, without any peer knowing the full membership. FileNest uses the DHT only through these two operations.

4 System Architecture

FileNest runs on a set of peers connected by a peer-to-peer network. Its design separates four concerns: representing content as embeddings (Section 4.1), organizing the embeddings



(a) Cross-section through 0, t, e

(b) The cap on the unit sphere

(c) Chart view: distance = angle

Figure 2: Angular distance, cosine similarity, and caps. (a) In the plane through the origin, t and e, the similarity $\mathrm{sim}(e,t) = \cos\phi$ is the length of the projection of the unit vector e onto t. The cap $C(t,\alpha)$ (thick arc) contains the unit vectors within angle α of t: exactly those whose projection is at least $\theta = \cos\alpha$, i.e., those on or above the plane $\hat{t}^\top x = \theta$. Here e lies inside the cap and e' outside. (b) In $\mathbb{R}^n$ the same set is a spherical cap around t. (c) The chart view used in Figures 4–7: the sphere's surface drawn flat, with distance on the page standing for angular distance, so that a cap is a disc of radius α.

in a hierarchical index (Sections 4.2 and 4.4), assigning index state to responsible peers (Section 4.3), and propagating queries through the hierarchy (Section 4.7). Figure 1 shows the components of the system and its two end-to-end operations, indexing a file and resolving a query, on a small example; the remainder of this section defines each step.

4.1 System Model

Let $\mathcal{P} = \{p_1, p_2, \ldots, p_m\}$ be the set of peers in the FileNest network. Each peer $p \in \mathcal{P}$ has a unique identifier $\mathrm{PID}(p)$ in the DHT key space (Section 3.2) and an addressable endpoint through which it exchanges messages with other peers.

Let $\mathcal{F}_p$ be the set of content objects that peer p makes available for indexing, and $\mathcal{F} = \bigcup_{p\in\mathcal{P}} \mathcal{F}_p$. FileNest peers do not store content bytes. When a content object f is submitted at an origin peer, the peer forwards f to a decentralized, content-addressed storage layer outside the indexing network, such as IPFS [3], which returns a content identifier $\mathrm{CID}(f)$ derived from the hash of f. FileNest routes $\mathrm{CID}(f)$ through the hierarchy, not f itself, and retrieving f requires a separate lookup of $\mathrm{CID}(f)$ in the storage layer. The index therefore holds only what is needed to locate and rank content, namely identifiers and embeddings, and no index node needs the content or a replica of it.

A common embedding function $E : \mathcal{F} \to \mathbb{R}^n$ maps a content object f to an embedding $e_f = E(f)$, and a query q to an embedding $e_q = E(q)$ in the same space. We treat E as a black box: the model specifies only its input-output behavior and does not require E to be computed by a single peer or service. Decentralizing the computation of E is orthogonal to the hierarchy and routing procedures and is not developed here. E may accept heterogeneous input modalities, provided the resulting embeddings lie in a common space suitable for similarity comparison, as with jointly trained image and text encoders [45].

4.2 Tagging-Vector Hierarchy

FileNest organizes embeddings in a hierarchy of maximum depth $D \in \mathbb{N}$, with levels $d \in \{1, \ldots, D\}$. At each depth d, the system maintains a set of tagging vectors $\mathcal{T}_d = \{t_{1,d}, \ldots, t_{K_d,d}\}$, where each $t_{i,d} \in \mathbb{R}^n$ is a routing point in the embedding space. A tagging vector $t_{i,d}$ carries a count $c_{i,d} \in \mathbb{N}$ of the embeddings assigned to it and a covering radius $\psi_{i,d} \in [0, \pi]$, an upper bound on the angle (Section 3.1) between $t_{i,d}$ and any embedding assigned to it, maintained as described in Section 4.6. We write $S(t_{i,d})$ for the set of embeddings assigned to $t_{i,d}$. Because every embedding indexed below $t_{i,d}$ is assigned to $t_{i,d}$ on its way down the hierarchy, $S(t_{i,d})$ is exactly the set of embeddings indexed in the subtree rooted at $t_{i,d}$. Finally, each tagging vector has a responsible peer $r(t_{i,d}) \in \mathcal{P}$, which maintains its routing state and is the next network hop for embeddings and queries assigned to it.

The depth-1 set contains $|\mathcal{T}_1| = K$ vectors, where K is a system parameter. They are derived during bootstrapping from the embeddings of a representative corpus $\mathcal{C}_0$; we evaluate both K and the derivation procedure empirically. The depth-1 tagging vectors and their responsible peers are then disseminated so that every peer starts with the same view of the root-level partition. At depths $d > 1$, tagging vectors are created during indexing as content arrives, so the number of tagging vectors at each depth is not fixed in advance.

4.3 Distributed Responsibility

The hierarchy determines which tagging vector an embedding selects, but not which peer stores that vector's routing state. FileNest assigns this responsibility through a DHT (Section 3.2). For a tagging vector $t_{i,d}$, let $t_{i,d}^{(1)}$ denote its value at creation, before any update of Section 4.6. Its key is

$$\kappa(t_{i,d}, d) = H(\mathrm{bytes}(t_{i,d}^{(1)}) \| d),$$

where H is a cryptographic hash function and $\|$ denotes concatenation, and its responsible peer is $r(t_{i,d}) = \text{DHTLookup}(\kappa(t_{i,d}, d))$. We derive the key from the creation-time value because $t_{i,d}$ changes with every assignment: a key computed from the current value would move responsibility to a different peer after each update, and peers holding the earlier key could no longer locate the routing state. The key is therefore fixed for the lifetime of the tagging vector and stored with it.

FileNest thus separates two mappings: semantic similarity maps an embedding to a tagging vector, $e \mapsto t$, and the DHT maps a tagging vector to a peer, $(t, d) \mapsto p$. The first decides which region an embedding belongs to, and the second decides where that region's routing state is kept; the DHT performs no similarity computation. Because keys are hashes, tagging vectors that are adjacent in the hierarchy receive unrelated keys, so the routing state of a subtree is spread across peers rather than concentrated on one (Figure 3).

4.4 Depth-Dependent Creation Threshold

At each depth d, indexing compares an embedding with the tagging vectors available at that depth and either assigns it to the nearest one or creates a new tagging vector, and with it a new responsibility assignment (Section 4.5). An angular creation threshold $\alpha_d \in (0, \pi)$, with corresponding similarity threshold $\theta_d = \cos \alpha_d$, governs this decision. The thresholds satisfy

$$\alpha_1 > \alpha_2 > \cdots > \alpha_D, \quad \text{i.e.,} \quad \theta_1 < \theta_2 < \cdots < \theta_D.$$

An embedding must therefore lie closer to an existing tagging vector to join it at deeper depths: regions are wide near the root and become progressively narrower, so each depth refines the partition of the depth above it (Figure 4). Stated as a similarity, the threshold *increases* with depth.

Rather than choosing D independent values, we derive the schedule from two endpoints and a constant refinement ratio.

Upper endpoint. We set the depth-1 threshold from the bootstrap corpus $\mathcal{C}_0$ from which $\mathcal{T}_1$ is derived,

$$\alpha_1 = Q_\eta\left(\left\{\min_{t \in \mathcal{T}_1} \phi\big(E(f), t\big) \;:\; f \in \mathcal{C}_0\right\}\right),$$

where Q_η denotes the empirical η-quantile. With η close to 1 (for example $\eta = 0.95$), content resembling the bootstrap corpus joins an existing depth-1 region and only content unlike it creates a new depth-1 tagging vector. Deriving α_1 from data, rather than fixing a numerical value, accounts for the fact that the range of cosine similarities produced by an embedding function differs considerably between embedding models.

Lower endpoint. We set the depth-D threshold equal to the provisioned query radius β_0, the angular match radius of the queries for which the deployment is dimensioned (Section 4.7): $\alpha_D = \beta_0$, with $\beta_0 < \alpha_1$. A query retrieves every embedding within its match radius. When depth-D regions are narrower than that radius, the matches of a single query are divided among more depth-D tagging vectors, each with its own responsible peer, and the query must contact each of them; the additional messages do not change the result. When depth-D regions are much wider than the radius, each contacted region contributes more non-matching embeddings that are discarded only after transfer, and covering radii grow, weakening the pruning of Section 4.7. Setting $\alpha_D = \beta_0$ balances the two effects at the provisioned radius.

Interpolation. Between the endpoints, the threshold decreases geometrically,

$$\alpha_d = \alpha_1 \gamma^{d-1}, \qquad \gamma = \left(\frac{\alpha_D}{\alpha_1}\right)^{1/(D-1)} \in (0, 1),$$

for $D \geq 2$. Each depth narrows regions by the same factor γ relative to the depth above, so no single depth carries a disproportionate share of the refinement, and hence of the fan-out, between the root and the deepest depth. Successive levels whose covering radii shrink by a constant factor are also the organizing principle of the cover tree [4]. The three parameters (η, β_0, D) fully determine the schedule.

Creation rule. Let p be the peer performing the routing step at depth d (the origin peer for $d = 1$, and for $d > 1$ the responsible peer of the depth-$(d-1)$ tagging vector selected in the previous step), and let $\mathcal{T}_d(p) \subseteq \mathcal{T}_d$ be the tagging vectors available to it. For an embedding e, let

$$i^* = \arg\min_i \phi(e, t_{i,d}),$$

with exact ties broken in favor of the smaller DHT key $\kappa(t_{i,d}, d)$ (Section 4.3), so that every routing peer resolves a tie identically. A new tagging vector, and with it a new responsible peer, is created at depth d if and only if no available tagging vector lies within α_d of e (Figure 4a):

$$\mathcal{T}_d(p) = \emptyset \quad \text{or} \quad \phi(e, t_{i^*,d}) > \alpha_d \iff \text{sim}(e, t_{i^*,d}) < \theta_d.$$

Otherwise, e is assigned to $t_{i^*,d}$. At each depth this is the nearest-center variant of the leader clustering algorithm [26], with threshold α_d. Every embedding assigned to a depth-d tagging vector therefore lay within α_d of that vector at the time of assignment; the covering radius of Section 4.6 records how far the movement of the vector subsequently loosens this bound.

Overlaps and gaps. The creation regions of sibling tagging vectors, the caps of radius α_d around them, are acceptance regions, not a partition of the parent's region (Figure 4a).

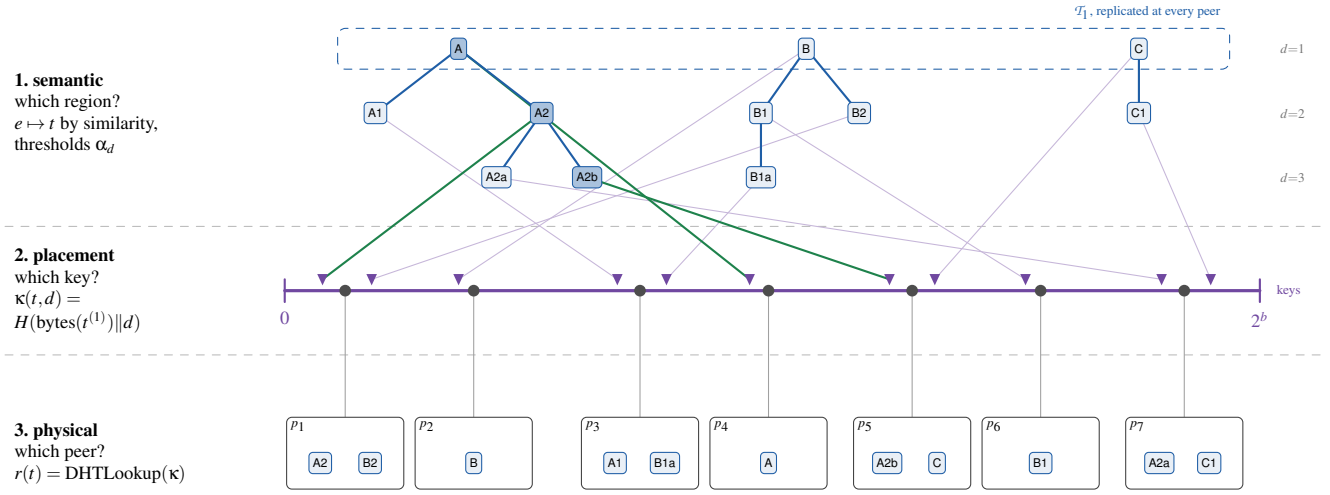
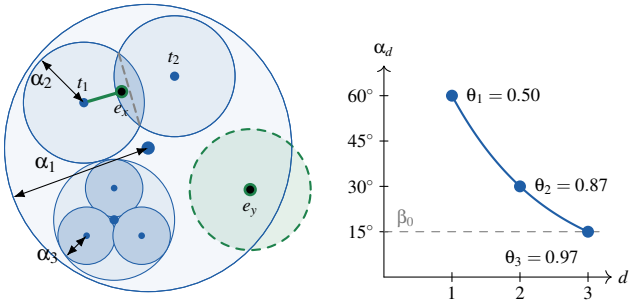


Figure 3: The mappings $e \mapsto t \mapsto p$ for the example of Figure 1. (1) Semantic: tagging vectors form the hierarchy, and $\mathcal{T}_1$ is replicated at every peer. (2) Placement: each tagging vector is hashed, from its creation-time value and its depth, to a key in the DHT key space of b-bit keys (drawn as a line). (3) Physical: each key is held by the peer whose identifier is closest under the DHT's distance metric. Neighboring regions (A, A1, A2) land on unrelated peers (p_4, p_3, p_1), so no peer holds a whole subtree. The highlighted path A→A2→A2b is the indexing path of Figure 1(a).



(a) Overlaps and gaps

(b) $\alpha_d = \alpha_1 \gamma^{d-1}$, $\gamma = \frac{1}{2}$

Figure 4: Depth-dependent creation thresholds. (a) A depth-d tagging vector accepts embeddings within α_d, and regions narrow at each depth (radii drawn schematically). Sibling regions can overlap: e_x lies in the regions of both t_1 and t_2 and is assigned to the nearer, t_1 (the dashed line is the nearest-center boundary). They can also leave gaps: e_y is farther than α_d from every child and becomes a new child (dashed), whose region crosses the parent's boundary: child regions need not lie inside their parent's region, and only the covering radius is used for pruning. (b) The schedule of Table 1: α_1 is set from the bootstrap corpus, $\alpha_D = \beta_0$, and intermediate depths interpolate geometrically; the similarity threshold $\theta_d = \cos \alpha_d$ rises with depth.

The creation rule places a new sibling more than α_d from every existing one, but not necessarily more than $2\alpha_d$, so the caps of two siblings may overlap, and the movement of tagging vectors under Section 4.6 may increase the overlap. Because assignment selects the nearest center before testing the threshold, an embedding in an overlap is never ambiguous: the set of embeddings a child $t_{i,d}$ can receive is

$$\left\{ e : \phi(e, t_{i,d}) \le \alpha_d \text{ and } \phi(e, t_{i,d}) \le \phi(e, t_{j,d}) \text{ for all } j \right\},$$

the nearest-center (Voronoi) cell of $t_{i,d}$ truncated at α_d. These sets are disjoint apart from ties, which the tie-breaking rule resolves. Conversely, the caps need not cover the parent's region: an embedding falling in a gap between them is farther than α_d from every sibling and creates a new one. Nor need the caps lie inside the parent's region. A child is created at an embedding within α_{d-1} of its parent, so its cap can reach up to $\alpha_{d-1} + \alpha_d$ from the parent's center (e_y in Figure 4a), and later movement of either vector can change the overhang. Requiring containment would leave content near the edge of a parent's region with no child to join. The overhang is harmless: an embedding reaches a child only after it has been assigned to the parent, so the part of a child's cap outside the parent's region receives no embeddings at assignment time, and the child's effective region is its cap intersected with the parent's. Regions are therefore created where content arrives rather than laid out in advance, and every embedding is routed to exactly one tagging vector at every depth. As tagging vectors move and siblings are added, the cells shift, so an embedding indexed earlier may later lie nearer to a sibling than to the vector it was assigned to. This does not affect query processing, which relies only on the covering radius of each tagging vector (Sections 4.6 and 4.7) and never assumes that regions are disjoint, exhaustive, or nested.

4.5 Indexing Procedure

Consider a content object f introduced by an origin peer p_o. As described in Section 4.1, p_o first submits f to the storage layer, obtains $\mathrm{CID}(f)$, and computes $e_f = E(f)$; it does not retain the bytes of f. At each depth d, the routing peer p applies the creation rule of Section 4.4 to e_f and $\mathcal{T}_d(p)$. If e_f lies within α_d of its nearest tagging vector $t_{i^*,d}$, p assigns e_f to it, updates its state (Section 4.6), and forwards the content metadata, consisting of $\mathrm{CID}(f)$, e_f, and p_o, to $r(t_{i^*,d})$. Otherwise, p creates a new tagging vector $t_{\mathrm{new},d} = e_f$ with count $c_{\mathrm{new},d} = 1$ and covering radius $\psi_{\mathrm{new},d} = 0$, since the only embedding assigned to it is the one it was created from, and forwards the metadata to its responsible peer $r(t_{\mathrm{new},d}) = \mathrm{DHTLookup}(H(\mathrm{bytes}(e_f) \| d))$.

For $d < D$, the receiving peer repeats the step at depth $d+1$ with threshold α_{d+1}. At depth D, the reached leaf peer stores $\mathrm{CID}(f)$ together with e_f and p_o; the bytes of f remain only in the storage layer.

4.6 Tagging-Vector Update

Let $t_{i,d}^{(c)}$ denote a tagging vector after c assignments. When an embedding e is assigned to it, we increment its count to $c+1$ and move the vector to the running mean of its assigned embeddings,

$$t'_{i,d} = t_{i,d}^{(c)} + \frac{1}{c+1}\left(e - t_{i,d}^{(c)}\right),$$

the sequential mean update of MacQueen's formulation of k-means [36]. The influence of each new embedding therefore decreases as the count grows.

Covering radius. The covering radius maintains the invariant

$$\phi(t_{i,d}, e) \le \psi_{i,d} \qquad \text{for every } e \in S(t_{i,d}). \tag{1}$$

At creation, $t_{i,d} = e_f$ and $S(t_{i,d}) = \{e_f\}$, so $\psi_{i,d} = 0$ satisfies (1). When an embedding e is assigned, let $\Delta = \phi(t_{i,d}^{(c)}, t'_{i,d})$ denote the angle through which the vector moves. We update the covering radius as

$$\psi'_{i,d} = \max\left\{\psi_{i,d} + \Delta,\ \phi\left(t'_{i,d}, e\right)\right\}.$$

The second term covers the new embedding. For an embedding e_j assigned earlier, the triangle inequality and (1) before the update give

$$\phi(t'_{i,d}, e_j) \le \phi(t'_{i,d}, t_{i,d}^{(c)}) + \phi(t_{i,d}^{(c)}, e_j) \le \Delta + \psi_{i,d},$$

so the first term covers every earlier embedding, and (1) holds after the update (Figure 5). Because the vector moves a fraction $1/(c+1)$ of the way toward each new embedding, Δ diminishes as c grows, and the radius of an established tagging vector grows only slowly with further assignments. Since every assigned embedding lay within α_d of the vector when assigned (Section 4.4), the radius exceeds α_d by at most the total angle through which the vector has moved.

The update yields an upper bound that never decreases. It may be replaced at any time by a tighter value that also satisfies (1): at depth D, by the exact maximum of $\phi(t_{i,D}, e)$ over the stored embeddings; and at depth $d < D$, where $S(t_{i,d})$ is the union of $S(t_{j,d+1})$ over the children $t_{j,d+1}$ of $t_{i,d}$, by

$$\psi_{i,d} \leftarrow \min\left\{\psi_{i,d},\ \max_j\left(\phi(t_{i,d}, t_{j,d+1}) + \psi_{j,d+1}\right)\right\},$$

which follows from the triangle inequality applied through each child.

A peer may apply the update asynchronously: forwarding the content metadata to the next depth need not wait for it to complete.

4.7 Query Processing

A query q is embedded as $e_q = E(q)$ and specifies a match threshold $\theta_q \in [-1, 1]$, with angular match radius $\beta_q = \arccos\theta_q$; when no threshold is specified, $\beta_q = \beta_0$. The answer to the query is the set of indexed content whose embedding lies within β_q of the query,

$$\mathcal{M}(q) = \{f : \phi(e_q, e_f) \le \beta_q\} = \{f : \mathrm{sim}(e_q, e_f) \ge \theta_q\}.$$

The query specifies only this single threshold, from which the cutoffs that prune the hierarchy are derived as follows.

Pruning cutoff. A routing peer at depth d evaluates each available tagging vector $t_{i,d}$ and explores its subtree if and only if

$$\phi(e_q, t_{i,d}) \le \beta_q + \psi_{i,d},$$

or, stated as a similarity cutoff,

$$\mathrm{sim}(e_q, t_{i,d}) \ge \theta_{i,d}(q) = \cos\left(\min\{\pi,\ \beta_q + \psi_{i,d}\}\right).$$

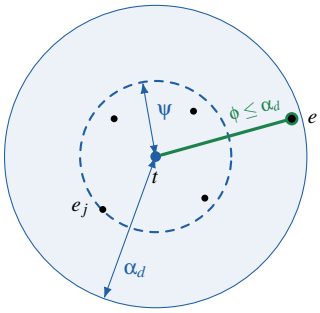
Only tagging vectors in the active set

$$A_d(q) = \left\{t_{i,d} \;\middle|\; \mathrm{sim}(e_q, t_{i,d}) \ge \theta_{i,d}(q)\right\}$$

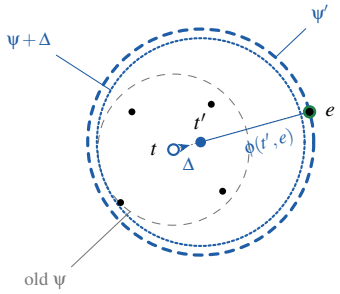
generate outgoing query messages. The cutoff is a property of each subtree rather than of the depth alone: a subtree whose embeddings are spread widely around its tagging vector is explored more permissively than a compact one. This is the covering-radius pruning of metric trees such as the M-tree [8], applied to the angular metric over a distributed hierarchy.

Lemma 1 (Safe pruning)**.** *If $\phi(e_q, t_{i,d}) > \beta_q + \psi_{i,d}$, then no embedding in the subtree rooted at $t_{i,d}$ lies within β_q of e_q.*

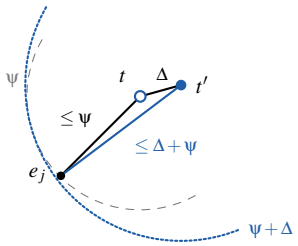
Proof. For any $e \in S(t_{i,d})$, the triangle inequality and (1) give $\phi(e_q, e) \ge \phi(e_q, t_{i,d}) - \phi(t_{i,d}, e) > (\beta_q + \psi_{i,d}) - \psi_{i,d} = \beta_q$. □



(a) e is assigned to t ($c = 4$)



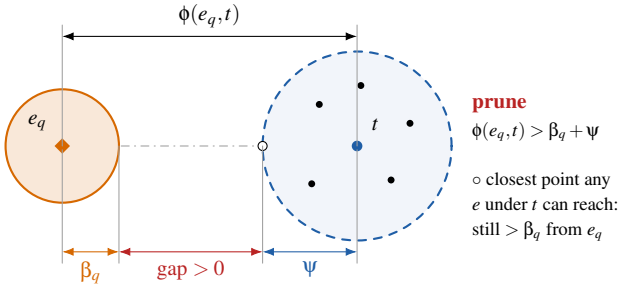
(b) t moves by Δ; $\psi' = \max\{\psi + \Delta, \phi(t', e)\}$



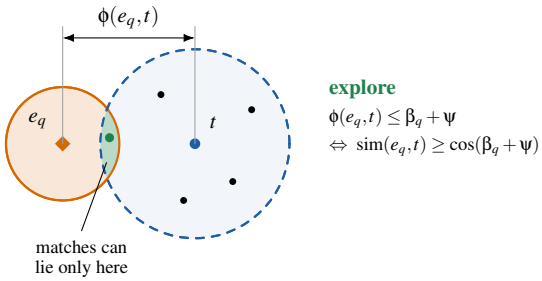
$\phi(t', e_j) \le \phi(t', t) + \phi(t, e_j) \le \Delta + \psi$

(c) Every earlier e_j stays covered

Figure 5: Tagging-vector update and covering radius. (a) Before the update, t has $c = 4$ assigned embeddings, all within its covering radius ψ (dashed); the arriving embedding e lies within α_d and is assigned. (b) The vector moves a fraction $1/(c+1)$ of the way toward e, through angle Δ. The disc of radius $\psi + \Delta$ around t' (dotted) contains the old covering disc and so covers every earlier embedding, while the disc of radius $\phi(t', e)$ covers e. The new radius ψ' (bold dashed) is the larger of the two, here $\phi(t', e)$. (c) The triangle inequality behind the first term, shown for the earlier embedding e_j.



(a) Disjoint: no match below t



(b) Overlapping: t may hold a match

Figure 6: The pruning test of Lemma 1, in the chart view. (a) The query disc (radius β_q around e_q) and the covering disc (radius ψ around t) are disjoint: even the closest point the subtree could contain (○) is farther than β_q from the query, so the subtree receives no message. (b) The discs overlap and the subtree is explored; any match lies in the overlap (shaded).

Geometrically, the test checks whether the query disc and the covering disc overlap (Figure 6).

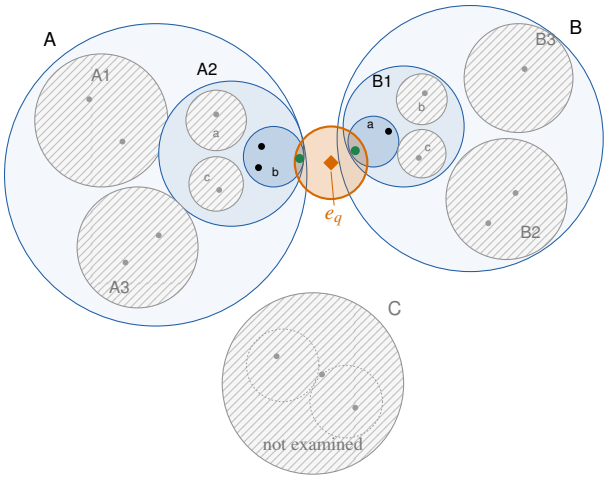
At depth 1, the client evaluates the query against $\mathcal{T}_1$ and forwards it to the responsible peer of each active tagging vector. At each subsequent depth, every reached routing peer performs the same operation over its available tagging vectors. At depth D, a reached peer evaluates the query against the embeddings stored at its leaf and returns the content identifiers, origin peers, and similarity scores of those satisfying $\phi(e_q, e_f) \le \beta_q$. The client aggregates the responses and ranks the results by their similarity to the query.

d	α_d	$\theta_d = \cos\alpha_d$ create if below	$\cos(\beta_q + \alpha_d)$ prune if below
1	60°	0.500	0.259
2	30°	0.866	0.707
3	15°	0.966	0.866

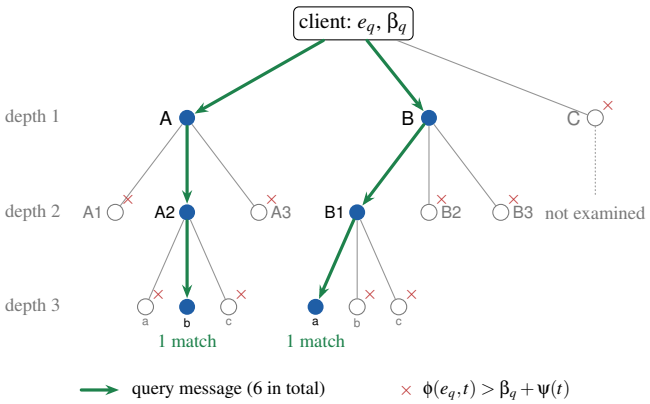
Table 1: Illustrative schedule for $D = 3$, $\alpha_1 = 60°$ and $\beta_0 = 15°$ ($\gamma = 1/2$), for a query at the provisioned radius $\beta_q = \beta_0$ ($\theta_q \approx 0.966$) and covering radii $\psi_{i,d} = \alpha_d$. Leaf entries are matched at θ_q. Actual values of α_1 and β_0 depend on the embedding model and the deployment.

A content object in $\mathcal{M}(q)$ is missed only if one of its ancestors fails the pruning test, and by Lemma 1 no ancestor of a match fails it; conversely, the leaf test returns nothing outside $\mathcal{M}(q)$. The procedure therefore returns exactly $\mathcal{M}(q)$: pruning discards only subtrees that contain no match. The guarantee holds with respect to the tagging-vector state used by the routing peers. When that state lags recent assignments, because of the asynchronous update of Section 4.6 or replica propagation in Section **??**, recently indexed content may be missed until the state catches up.

Behavior across depths. Because $\psi_{i,d}$ can exceed α_d only by the movement of $t_{i,d}$, the cutoff at depth d is approximately $\cos(\beta_q + \alpha_d)$: permissive near the root, where regions are wide, and increasingly strict with depth, reaching approximately $\cos(\beta_q + \beta_0)$ at the deepest routing step. Table 1 gives an example, and Figure 7 traces a complete query through a



(a) Regions, query disc, and matches



(b) The same query as messages

Figure 7: A query over a hierarchy with $D = 3$. (a) Depth-1 regions A, B, C, their depth-2 children (A1, ...), and depth-3 leaves (a, b, c), each drawn as its covering disc. A subtree is explored exactly when its covering disc overlaps the query disc (orange). Hatched subtrees fail the test and are never contacted, and nothing below a pruned vector (C) is examined. The two embeddings inside the query disc (green) form the answer $\mathcal{M}(q)$. (b) The same traversal as messages: the client tests the depth-1 vectors locally, each reached peer tests its own children, and six messages reach the two leaves that hold matches.

three-depth hierarchy. Creation and pruning thresholds thus tighten together with depth, but they are distinct: creation compares an embedding with a region's center within angle α_d, whereas pruning must account for every embedding the region may contain together with the query's own radius, and therefore allows angle $\beta_q + \psi_{i,d} \approx \beta_q + \alpha_d$.

4.8 Peer State and Messages

Figure 8 summarizes the state a single peer maintains and the messages it exchanges. Every peer holds its DHT routing table and the replicated root view $\mathcal{T}_1$. In addition, for each tagging vector whose key it is responsible for, as primary or as replica (Section **??**), it holds that vector's state together with either the table of its children, which the creation rule and the pruning test both consult, or, at depth D, the indexed entries. No peer holds the complete index, and no peer holds the bytes of any content object.

5 Design Extensions

6 Performance Analysis

7 Security Analysis and Discussion

8 Conclusion and Future Work

Reproducibility

The pipeline, the fault injector, the per-analysis records including every model output verbatim, and the scripts that generate every table in this paper are publicly available at: https://github.com/SPriTLab-CSE-IITJ/FileNest

Acknowledgment

The authors used a large language model for drafting and grammatical revision of the text in this paper. All experiments, results, and analysis are the authors' own. This work was supported by the Indian Institute of Technology Jodhpur, India under the Research Initiation Grant (RIG) Program (Grant No. I/I/RIG/SKM/20250216).

References

[1] Access Now. Rising repression meets global resistance: Internet shutdowns in 2025. #KeepItOn report, https://www.accessnow.org/wp-content/uploads/2026/03/KeepItOn-Internet-Shutdowns-202

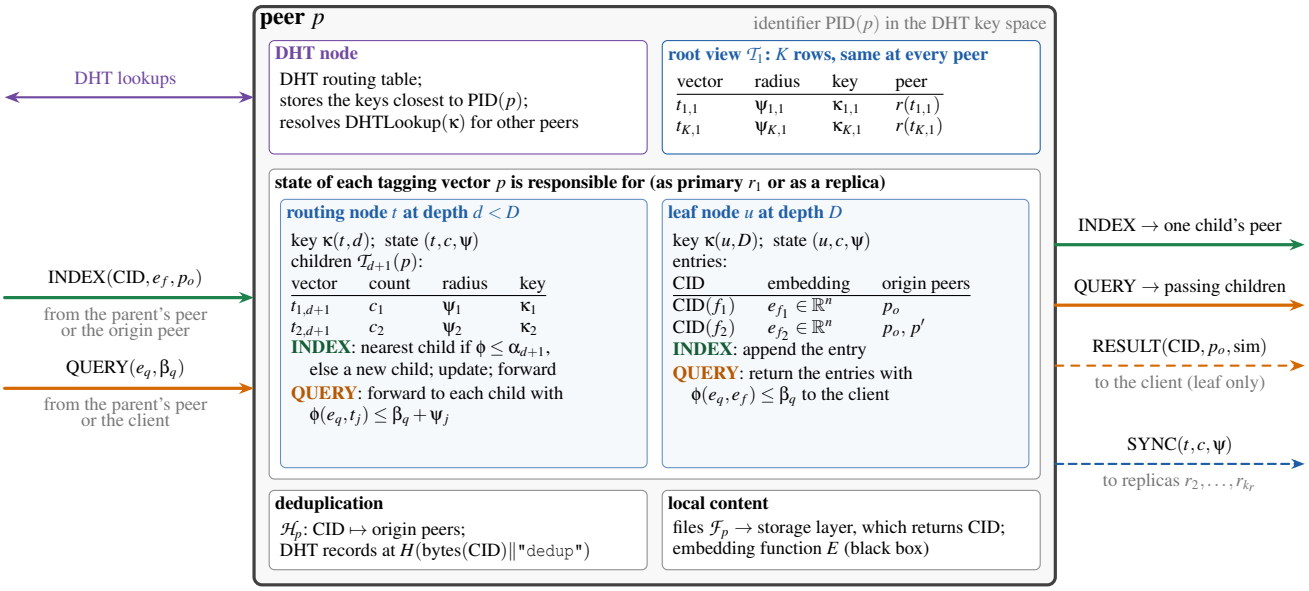


Figure 8: State and messages of a single peer p. INDEX and QUERY arrive from the parent's peer, or from the origin peer or client at depth 1. A peer forwards INDEX to exactly one child's peer and QUERY to every child that passes the pruning test; leaves return RESULT messages directly to the client; and SYNC carries updates of a tagging vector's state to its replicas (Section **??**).

5-Annual-Report.pdf, Mar. 2026. Accessed October 2026.

[2] ARSO PROJECT. Sonar: A peer-to-peer content database and search engine. https://github.com/arso-project/sonar. Accessed October 2026.

[3] BENET, J. IPFS – content addressed, versioned, P2P file system. arXiv preprint arXiv:1407.3561, July 2014.

[4] BEYGELZIMER, A., KAKADE, S., AND LANGFORD, J. Cover trees for nearest neighbor. In *Proceedings of the 23rd International Conference on Machine Learning (ICML 2006)* (Pittsburgh, PA, USA, 2006), vol. 148 of *ACM International Conference Proceeding Series*, ACM, pp. 97–104.

[5] BLOOM, B. H. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM 13*, 7 (July 1970), 422–426.

[6] BRODER, A., AND MITZENMACHER, M. Network applications of Bloom filters: A survey. *Internet Mathematics 1*, 4 (2004), 485–509.

[7] CIACCIA, P., AND PATELLA, M. PAC nearest neighbor queries: Approximate and controlled search in high-dimensional and metric spaces. In *Proceedings of the 16th International Conference on Data Engineering (ICDE 2000)* (San Diego, CA, USA, 2000), IEEE Computer Society, pp. 244–255.

[8] CIACCIA, P., PATELLA, M., AND ZEZULA, P. M-tree: An efficient access method for similarity search in metric spaces. In *Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB '97)* (Athens, Greece, 1997), Morgan Kaufmann, pp. 426–435.

[9] CLARKE, I., SANDBERG, O., WILEY, B., AND HONG, T. W. Freenet: A distributed anonymous information storage and retrieval system. In *Designing Privacy Enhancing Technologies: International Workshop on Design Issues in Anonymity and Unobservability* (Berkeley, CA, USA, 2001), vol. 2009 of *Lecture Notes in Computer Science*, Springer, pp. 46–66.

[10] COURT OF JUSTICE OF THE EUROPEAN UNION. Judgment of 13 May 2014, Google Spain SL and Google Inc. v Agencia Española de Protección de Datos (AEPD) and Mario Costeja González, Case C-131/12. ECLI:EU:C:2014:317, May 2014.

[11] COURT OF JUSTICE OF THE EUROPEAN UNION. Judgment of 10 September 2024, Google and Alphabet v Commission (Google Shopping), Case C-48/22 P. ECLI:EU:C:2024:726, Sept. 2024.

[12] COX, J. Bing censors image search for 'Tank Man' even in US. Motherboard (VICE), https://www.vice.com/en/article/bing-censors-tank-man/, June 2021. Accessed October 2026.

[13] DABEK, F., KAASHOEK, M. F., KARGER, D., MORRIS, R., AND STOICA, I. Wide-area cooperative storage with CFS. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP '01)* (Banff, Alberta, Canada, Oct. 2001), ACM, pp. 202–215.

[14] DANIEL, E., AND TSCHORSCH, F. IPFS and friends: A qualitative comparison of next generation peer-to-peer data networks. *IEEE Communications Surveys & Tutorials 24*, 1 (2022), 31–52.

[15] DINGLEDINE, R., FREEDMAN, M. J., AND MOLNAR, D. The Free Haven project: Distributed anonymous storage service. In *Designing Privacy Enhancing Technologies: International Workshop on Design Issues in Anonymity and Unobservability* (Berkeley, CA, USA, 2001), vol. 2009 of *Lecture Notes in Computer Science*, Springer, pp. 67–95.

[16] DOUCEUR, J. R., ADYA, A., BOLOSKY, W. J., SIMON, D., AND THEIMER, M. Reclaiming space from duplicate files in a serverless distributed file system. In *Proceedings of the 22nd International Conference on Distributed Computing Systems (ICDCS '02)* (Vienna, Austria, July 2002), IEEE Computer Society, pp. 617–624.

[17] DOULKERIDIS, C., VLACHOU, A., KOTIDIS, Y., AND VAZIRGIANNIS, M. Peer-to-peer similarity search in metric spaces. In *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB 2007)* (Vienna, Austria, 2007), ACM, pp. 986–997.

[18] DRUMMOND, D. A new approach to China. Official Google Blog, https://googleblog.blogspot.com/2010/01/new-approach-to-china.html, Jan. 2010. Accessed October 2026.

[19] DRUMMOND, D. A new approach to China: An update. Official Google Blog, https://googleblog.blogspot.com/2010/03/new-approach-to-china-update.html, Mar. 2010. Accessed October 2026.

[20] EPSTEIN, R., AND ROBERTSON, R. E. The search engine manipulation effect (SEME) and its possible impact on the outcomes of elections. *Proceedings of the National Academy of Sciences 112*, 33 (2015), E4512–E4521.

[21] EUROPEAN COMMISSION. Commission decision of 27 June 2017 relating to proceedings under Article 102 TFEU and Article 54 of the EEA Agreement (Case AT.39740 – Google Search (Shopping)). C(2017) 4444 final, June 2017.

[22] FILASTÒ, A., AND APPELBAUM, J. OONI: Open observatory of network interference. In *Proceedings of the 2nd USENIX Workshop on Free and Open Communications on the Internet (FOCI '12)* (Bellevue, WA, USA, 2012), USENIX Association.

[23] FREEDOM HOUSE. Freedom on the net 2025: An uncertain future for the global internet. https://freedomhouse.org/report/freedom-net/2025/uncertain-future-global-internet, Nov. 2025. Accessed October 2026.

[24] GHEMAWAT, S., GOBIOFF, H., AND LEUNG, S.-T. The Google file system. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03)* (Bolton Landing, NY, USA, Oct. 2003), ACM, pp. 29–43.

[25] GREGORIADIS, M., CHOTKAN, R. M., NEAGUE, P., AND POUWELSE, J. SwarmSearch: Decentralized search engine with self-funding economy. In *2025 IEEE 50th Conference on Local Computer Networks (LCN)* (Sydney, Australia, 2025), IEEE, pp. 1–10.

[26] HARTIGAN, J. A. *Clustering Algorithms*. Wiley Series in Probability and Mathematical Statistics. John Wiley & Sons, New York, NY, USA, 1975.

[27] HERRMANN, M., NING, K.-C., DIAZ, C., AND PRENEEL, B. Description of the YaCy distributed web search engine. Technical report, KU Leuven ESAT/COSIC and iMinds, Leuven, Belgium, 2014.

[28] INTRONA, L. D., AND NISSENBAUM, H. Shaping the Web: Why the politics of search engines matters. *The Information Society 16*, 3 (2000), 169–185.

[29] IPFS-SEARCH.COM. ipfs-search: Search engine for the InterPlanetary File System. https://github.com/ipfs-search/ipfs-search, 2023. Public service offline since mid-2023. Accessed October 2026.

[30] JÉGOU, H., DOUZE, M., AND SCHMID, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence 33*, 1 (2011), 117–128.

[31] JOHNSON, J., DOUZE, M., AND JÉGOU, H. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data 7*, 3 (2021), 535–547.

[32] KARPUKHIN, V., OĞUZ, B., MIN, S., LEWIS, P., WU, L., EDUNOV, S., CHEN, D., AND YIH, W.-T. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (Online, 2020), Association for Computational Linguistics, pp. 6769–6781.

[33] KNOCKEL, J., AND RUAN, L. Bada Bing, bada boom: Microsoft Bing's Chinese political censorship of autosuggestions in North America. Citizen Lab Research Report No. 156, University of Toronto, https://citizenlab.ca/research/bada-bing-bada-boom-microsoft-bings-chinese-political-censorship-autosuggestions-north-america/, May 2022. Accessed October 2026.

[34] KOLONIARI, G., AND PITOURA, E. Content-based routing of path queries in peer-to-peer systems. In *Advances in Database Technology – EDBT 2004: 9th International Conference on Extending Database Technology* (Heraklion, Crete, Greece, 2004), vol. 2992 of *Lecture Notes in Computer Science*, Springer, pp. 29–47.

[35] LI, M., ZHU, J., ZHANG, T., TAN, C., XIA, Y., ANGEL, S., AND CHEN, H. Bringing decentralized search to decentralized services. In *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI '21)* (2021), USENIX Association, pp. 331–347.

[36] MACQUEEN, J. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* (Berkeley, CA, USA, 1967), vol. 1, University of California Press, pp. 281–297.

[37] MALKOV, Y. A., AND YASHUNIN, D. A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence 42*, 4 (2020), 824–836.

[38] MAYMOUNKOV, P., AND MAZIÈRES, D. Kademlia: A peer-to-peer information system based on the XOR metric. In *Peer-to-Peer Systems: First International Workshop (IPTPS 2002), Revised Papers* (Cambridge, MA, USA, Mar. 2002), vol. 2429 of *Lecture Notes in Computer Science*, Springer, pp. 53–65.

[39] MUTHITACHAROEN, A., CHEN, B., AND MAZIÈRES, D. A low-bandwidth network file system. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP '01)* (Banff, Alberta, Canada, Oct. 2001), ACM, pp. 174–187.

[40] NEAGUE, P., GREGORIADIS, M., AND POUWELSE, J. De-DSI: Decentralised differentiable search index. In *Proceedings of the 4th Workshop on Machine Learning and Systems (EuroMLSys '24)* (Athens, Greece, 2024), ACM, pp. 134–143.

[41] NIAKI, A. A., CHO, S., WEINBERG, Z., HOANG, N. P., RAZAGHPANAH, A., CHRISTIN, N., AND GILL, P. ICLab: A global, longitudinal Internet censorship measurement platform. In *Proceedings of the 2020 IEEE Symposium on Security and Privacy (SP)* (2020), IEEE, pp. 135–151.

[42] NOVAK, D., AND ZEZULA, P. M-Chord: A scalable distributed similarity search structure. In *Proceedings of the 1st International Conference on Scalable Information Systems (InfoScale '06)* (Hong Kong, 2006), vol. 152 of *ACM International Conference Proceeding Series*, ACM. Article 19.

[43] OTTE, P., DE VOS, M., AND POUWELSE, J. TrustChain: A Sybil-resistant scalable blockchain. *Future Generation Computer Systems 107* (2020), 770–780.

[44] POUWELSE, J. A., GARBACKI, P., WANG, J., BAKKER, A., YANG, J., IOSUP, A., EPEMA, D. H. J., REINDERS, M. J. T., VAN STEEN, M. R., AND SIPS, H. J. TRIBLER: A social-based peer-to-peer system. *Concurrency and Computation: Practice and Experience 20*, 2 (2008), 127–138.

[45] RADFORD, A., KIM, J. W., HALLACY, C., RAMESH, A., GOH, G., AGARWAL, S., SASTRY, G., ASKELL, A., MISHKIN, P., CLARK, J., KRUEGER, G., AND SUTSKEVER, I. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)* (2021), vol. 139 of *Proceedings of Machine Learning Research*, PMLR, pp. 8748–8763.

[46] RAMAN, R. S., SHENOY, P., KOHLS, K., AND ENSAFI, R. Censored Planet: An Internet-wide, longitudinal censorship observatory. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS '20)* (2020), ACM, pp. 49–66.

[47] RATAJ, M. Distributed Wikipedia mirror update. IPFS Blog, https://blog.ipfs.tech/2021-05-31-distributed-wikipedia-mirror-update/, May 2021. Accessed October 2026.

[48] RATNASAMY, S., FRANCIS, P., HANDLEY, M., KARP, R., AND SHENKER, S. A scalable content-addressable network. In *Proceedings of the ACM SIGCOMM 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '01)* (San Diego, CA, USA, Aug. 2001), ACM, pp. 161–172.

[49] REIMERS, N., AND GUREVYCH, I. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (Hong Kong, China, 2019),

Association for Computational Linguistics, pp. 3982–3992.

[50] ROWSTRON, A., AND DRUSCHEL, P. Storage management and caching in PAST, a large-scale, persistent peer-to-peer storage utility. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP '01)* (Banff, Alberta, Canada, Oct. 2001), ACM, pp. 188–201.

[51] STATCOUNTER. Search engine market share worldwide. Statcounter Global Stats, https://gs.statcounter.com/search-engine-market-share, Sept. 2026. All platforms, September 2026. Accessed October 2026.

[52] STOICA, I., MORRIS, R., KARGER, D., KAASHOEK, M. F., AND BALAKRISHNAN, H. Chord: A scalable peer-to-peer lookup service for Internet applications. In *Proceedings of the ACM SIGCOMM 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '01)* (San Diego, CA, USA, Aug. 2001), ACM, pp. 149–160.

[53] TANG, C., XU, Z., AND DWARKADAS, S. Peer-to-peer information retrieval using self-organizing semantic overlay networks. In *Proceedings of the 2003 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM '03)* (Karlsruhe, Germany, 2003), ACM, pp. 175–186.

[54] TAY, Y., TRAN, V. Q., DEHGHANI, M., NI, J., BAHRI, D., MEHTA, H., QIN, Z., HUI, K., ZHAO, Z., GUPTA, J., SCHUSTER, T., COHEN, W. W., AND METZLER, D. Transformer memory as a differentiable search index. In *Advances in Neural Information Processing Systems (NeurIPS)* (2022), vol. 35, Curran Associates, Inc., pp. 21831–21843.

[55] THE IPFS TEAM. Uncensorable Wikipedia on IPFS. IPFS Blog, https://blog.ipfs.tech/24-uncensorable-wikipedia/, May 2017. Accessed October 2026.

[56] TRAUTWEIN, D., RAMAN, A., TYSON, G., CASTRO, I., SCOTT, W., SCHUBOTZ, M., GIPP, B., AND PSARAS, Y. Design and evaluation of IPFS: A storage layer for the decentralized Web. In *Proceedings of the ACM SIGCOMM 2022 Conference* (Amsterdam, Netherlands, 2022), ACM, pp. 739–752.

[57] U.S. DISTRICT COURT FOR THE DISTRICT OF COLUMBIA. United States v. Google LLC, memorandum opinion. No. 1:20-cv-03010-APM, ECF No. 1033 (D.D.C. Aug. 5, 2024) (Mehta, J.), reported at 747 F. Supp. 3d 1, Aug. 2024.

[58] WALDMAN, M., AND MAZIÈRES, D. Tangler: A censorship-resistant publishing system based on document entanglements. In *Proceedings of the 8th ACM Conference on Computer and Communications Security (CCS '01)* (Philadelphia, PA, USA, 2001), ACM, pp. 126–135.

[59] WALDMAN, M., RUBIN, A. D., AND CRANOR, L. F. Publius: A robust, tamper-evident, censorship-resistant web publishing system. In *Proceedings of the 9th USENIX Security Symposium* (Denver, CO, USA, 2000), USENIX Association, pp. 59–72.

[60] WIKIMEDIA FOUNDATION. Access to Wikipedia restored in Turkey after more than two and a half years. https://wikimediafoundation.org/news/2020/01/15/access-to-wikipedia-restored-in-turkey-after-more-than-two-and-a-half-years/, Jan. 2020. Accessed October 2026.

[61] ZHANG, T., RAMAKRISHNAN, R., AND LIVNY, M. BIRCH: An efficient data clustering method for very large databases. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data (SIGMOD '96)* (Montreal, QC, Canada, 1996), ACM, pp. 103–114.

[62] ZITTRAIN, J., AND EDELMAN, B. Localized Google search result exclusions: Statement of issues and call for data. Berkman Center for Internet & Society, Harvard Law School, https://cyber.harvard.edu/filtering/google/, Oct. 2002. Accessed October 2026.

Notes